

Temat projektu: Gra SFML

Autor gry: Dominik Słomiński

Politechnika Gdańska, Wydział Elektroniki i Automatyki,

Kierunek: Robotyka, Automatyka i Systemy Sterowania,

Przedmiot: Informatyka II

Prowadzący: dr Robert Smyk

Data zakończenia: 14.01.2025

Cel projektu: Celem projektu było stworzenie prostej gry komputerowej 2D z wykorzystaniem biblioteki SFML (Simple and Fast Multimedia Library). Projekt ten powstał w ramach zajęć [Informatyka II, jako część praktycznego podejścia do nauki programowania przy użyciu graficznego interfejsu.

Zakres projektu obejmował:

- zaprojektowanie mechaniki gry,
- implementację podstawowych funkcjonalności, takich jak obsługa kolizji, animacje i sterowanie,
- opracowanie graficznego interfejsu użytkownika.

Rezultatem projektu jest w pełni działająca gra 2D, która pozwala graczowi na interakcję z wirtualnym światem. Realizacja projektu pozwoliła na zdobycie praktycznych umiejętności programistycznych, a także na zapoznanie się z procesami projektowania i implementacji prostych gier komputerowych.

Wymagania: Gra musi być funkcjonalna, kod powinien być zorganizowany w klasy, powinien zawierać elementy graficzne. Gracz powinien mieć możliwość kontroli animacji za pomocą klawiatury. Gra powinna mieć możliwość zapisu i odczytu danych poprzedniej gry.

Projekt gry:

○ **Mechanika gry:**

Celem gry jest przetrwanie jak najdłużej, unikanie kolizji z meteorami, po której gracz traci jedno życie. Meteory poruszają się po ekranie w losowych kierunkach. Po odbiciu od krawędzi ekranu ich kierunek zmienia się, co utrudnia przewidywanie trajektorii.

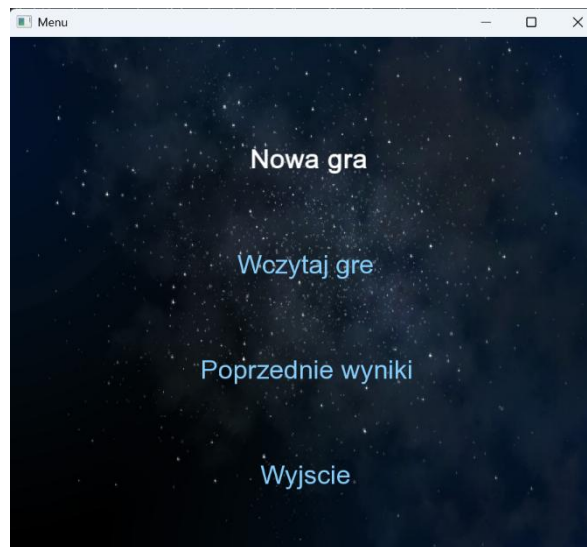
Gracz kontroluje statek za pomocą klawiszy W, A, S, D, umożliwiających poruszanie się w czterech kierunkach. Statek reaguje płynnie na polecenia, zapewniając intuicyjne sterowanie.

○ **Elementy gry:**

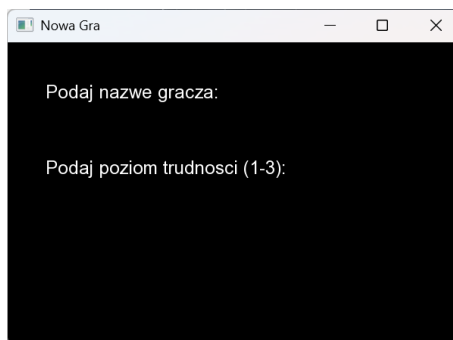
1. Gracz: Statek kosmiczny z możliwością sterowania klawiaturą w 2 wymiarach.
2. Przeszkody: Meteoryty pojawiające się w losowych miejscach, poruszające się w losowych kierunkach, odbijające się od krawędzi.
3. Okno gry: Pasek górny, na którym wyświetla się poziom gry, czas gry, zdobyte punkty. Dolny pasek, na którym wyświetla się nazwa gracza oraz nazwa autora gry.
4. Menu: menu, w którym wybiera się opcje gry.

- **Fazy gry:**

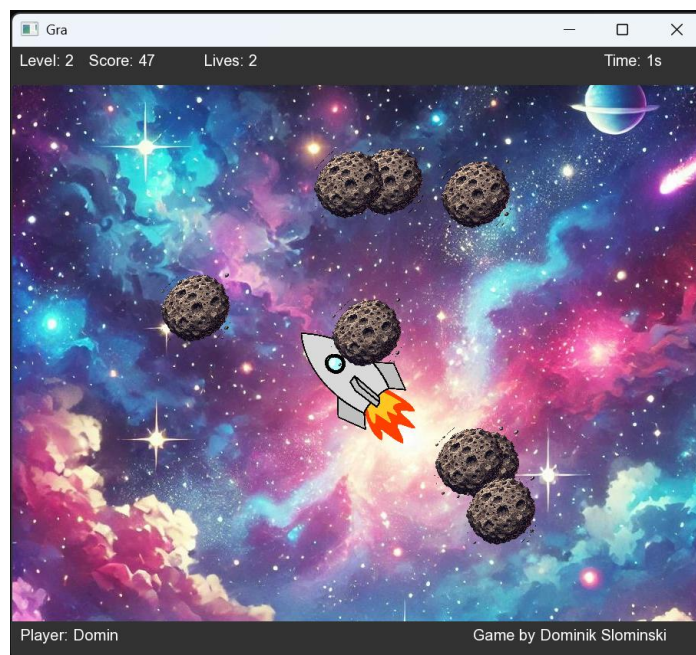
1. Menu: Daje możliwość wyboru opcji gry, nowa gra, wczytanie poprzedniej, niedokończonej gry, wyświetlenie poprzedniego wyniku zakończonej gry oraz opcja wyjście zamykająca grę.



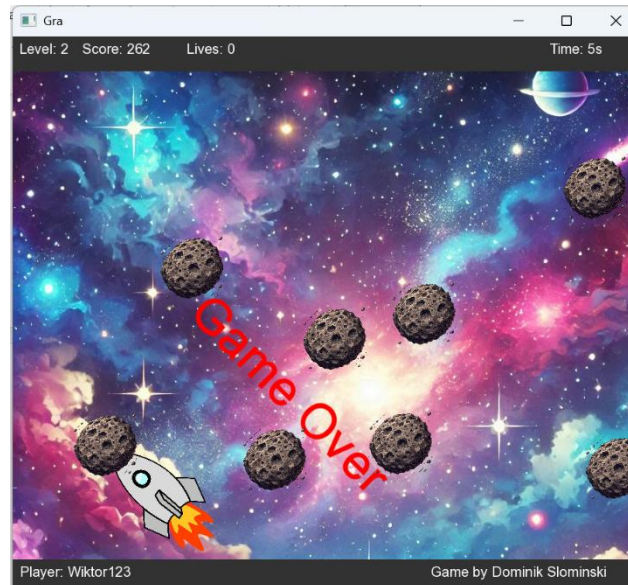
2. Okno nowej gry, służy do wprowadzania nazwy gracza oraz poziomu trudności



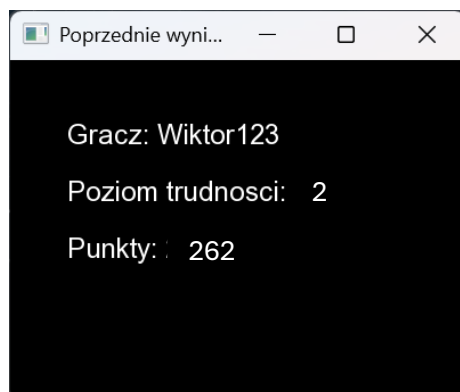
3. Właściwa gra, w której toczy się rozgrywka, z wszystkimi podanymi wcześniej elementami: okno gry, statek, meteoryty.



4. Koniec gry. Gra kończy się w momencie utraty wszystkich żyć przez gracza, następuje zatrzymanie ruchu meteorytów oraz wyświetla się duży napis: **Game Over**. W tym momencie następuje zapis wyniku gry do pliku.



5. Okno odczytu poprzedniego wyniku: umożliwia odczyt danych.

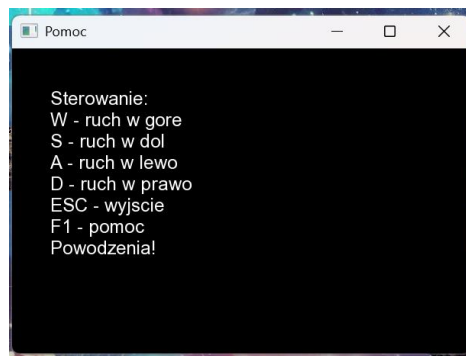


○ **Dodatkowe elementy**

- Okno wyjścia z gry: na ekranie pojawia się czarne tło z napisem „Czy na pewno chcesz opuścić grę?(T/N)”. Po wybraniu opcji T zamyka się gra, zapisuje aktualny stan gry, który można potem wczytać opcją „Wczytaj grę”. Gra rozpocznie się od momentu pojawienia się napisu.

Czy na pewno chcesz opuścić grę? (T/N)

- Okno pomocy: pojawia się po naciśnięciu F1, zawiera informacje jak grać.



Implementacja:

Projekt był budowany z wykorzystaniem technik programowania obiektowego. Charakteryzuje się hierarchiczną budową kodu, poszczególne klasy, funkcje mają konkretne zadania. Kod składa się z głównej pętli, w której jest tworzone okno menu oraz wywoływane poszczególne klasy i funkcje, zadeklarowane wcześniej, poza funkcją main():

- menu – odpowiada za wybór opcji menu, wyświetlanie opcji,
- nowa gra – okno służące do wprowadzania danych: nazwy gracza oraz poziomu, pojawia się w momencie wybrania opcji menu: Nowa gra,
- poprzednie wyniki – okno wyświetlające nazwę gracza, poziom i zdobyty wynik w poprzedniej grze, wczytuje te dane z pliku,
- runGame – funkcja odpowiadająca za okno gry, zawarta jest w niej cała dynamika gry.

Funkcja runGame zawiera poszczególne klasy:

- GameInterface – odpowiadająca za wygląd okna gry, zawiera zmienne stanu gry, czas, poziom, nazwę gracza, liczbę punktów oraz górny i dolny pasek, umożliwia zapis i odczyt danych gry z pliku.
- statek – odpowiada za gracza, animacje statku, położenie, prędkość, zapis i odczyt z pliku pozycji statku
- meteor – odpowiadająca za meteoryty, ruch, prędkość, animację meteorytów.
- GameOverScreen – odpowiada za koniec gry, za napis game over.
- save – zawiera funkcje odpowiedzialne za zapis i odczyt danych gry z pliku.

Dodatkowo w funkcji gry są używane funkcje windowHelp – odpowiedzialna za okno pomocy oraz okno_Exit – zajmująca się opuszczaniem gry. Funkcje te są zadeklarowane przed funkcją gry.

Testowanie:

Kod był testowany w większości metodą prób i błędów. Do testów zostały wykorzystane globalne zmienne pomocnicze, w celu łatwego doboru parametrów, np. prędkość statku czy meteorytów, szerokość interfejsu, margines odbicia meteorytów czy liczba żyć. Po zakończeniu testów zmienne te zostały zastąpione konkretnymi wartościami. Dodatkowo w celu sprawdzenia poprawności działania funkcji w fragmentach kodu wyświetlano aktualny status gry w konsoli, co pozwalało kontrolować działanie programu. Szczególnie było to pomocne przy zapisie i odczycie danych z pliku. Pozwalało to zweryfikować działanie funkcji oraz zlokalizować ewentualne błędy.

Wnioski:

Trudnościami podczas tworzenia projektu było zaplanowanie pracy, struktury kodu, wybranie, od czego zacząć. Dodatkowo problematyczne było zorganizowanie kodu, wybór klas oraz ogarnięcie zapisu danych gry do pliku i ich odczyt. Można byłoby dopracować kolizję, by była dokładniejsza, wprowadzić zmiany w związku z kolejnymi poziomami, dodać kulki pojawiające się w losowych miejscach, które dodawałyby życie albo punktów gry. Cała struktura gry mogłaby zostać lekko zmieniona. Projekt uważam za bardzo ciekawy, dający wiele możliwości rozwoju, ale wymagający dużych nakładów czasu i energii.